

T.N.S.V.T - Documentación Técnica Completa

Versión: 1.5.0 **Fecha:** 27 de Junio de 2026 **Backend:** Symfony 7.4 + PHP 8.4 **Frontend:** HTML/CSS/JS con Capacitor v8 + Android **Base de datos:** SQLite (Doctrine ORM)

1. Arquitectura General

La plataforma T.N.S.V.T (Trading Neuro-Spiritual Value Theory) es una aplicación híbrida que funciona como:

- Aplicación web progresiva (PWA):** Accesible desde cualquier navegador moderno
- App Android nativa:** Empaquetada con Capacitor v8, disponible como APK descargable

Stack tecnológico

Componente	Tecnología
Backend	Symfony 7.4 (PHP 8.4)
Frontend	Vanilla JS + CSS (sin frameworks)
Base de datos	SQLite vía Doctrine ORM
App wrapper	Capacitor v8 (Apache Cordova alternativa moderna)
Autenticación	Código de usuario (X-Game-Code) + Sesión web
HTTPS	Tailscale Serve (certificado automático Let's Encrypt)

Estructura de directorios

tnsvt-symfony/	
├─ assets/	# Código fuente JS/CSS (compilado por AssetMapper)
└─ app.js	# Frontend principal SPA (5577+ líneas)
└─ api.js	# Cliente HTTP para el backend
└─ styles/	# Estilos CSS
├─ src/	
└─ Controller/	# Controladores Symfony
└─ Api/	# Endpoints REST JSON
└─ Entity/	# Entidades Doctrine (ORM)
├─ templates/	# Twig templates
├─ public/	# Document root
└─ sw.js	# Service Worker (PWA)
└─ manifest.json	# PWA Manifest
└─ icons/	# Iconos PWA
├─ android/	# Proyecto Android nativo (Capacitor)
└─ migrations/	# Migraciones de base de datos

2. Funcionalidades Implementadas

2.1 Módulos principales

Módulo	Descripción	Estado
Manuscrito & Comunidad	Feed social con publicaciones, señales, categorías	✓
Chart en Vivo	Gráfico de velas con datos de Binance + TradingView	✓
Macroeconomía	Calendario económico con datos económicos	✓
Metodología 2 Steps	Sistema de trading estructurado	✓
Tareas Operativas	Gestión de tareas asignadas por admin	✓
Calendario	Calendario económico con respaldo local	✓
Trading Journal	Registro de trades con fotos y análisis	✓
Leaderboard	Ranking de traders por P&L	✓
Academia	Cursos y contenido educativo	✓
Chat	Mensajería entre usuarios	✓
Diario Personal	Cuaderno privado con cifrado AES-256-GCM	✓ NUEVO
Huella Digital	Desbloqueo biométrico del diario	✓ NUEVO

2.2 Sistema de pago

- **MercadoPago** - Integración de pagos argentinos
- **Binance Pay** - Pagos con criptomonedas

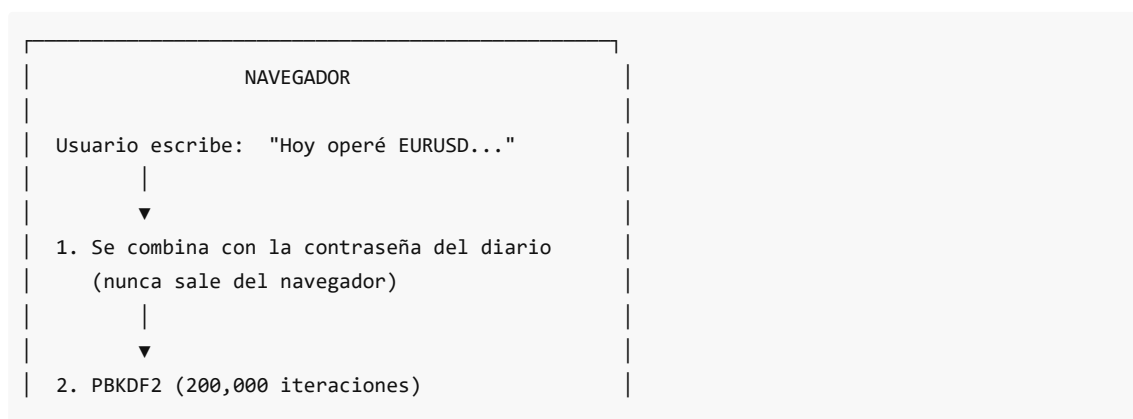
2.3 Notificaciones

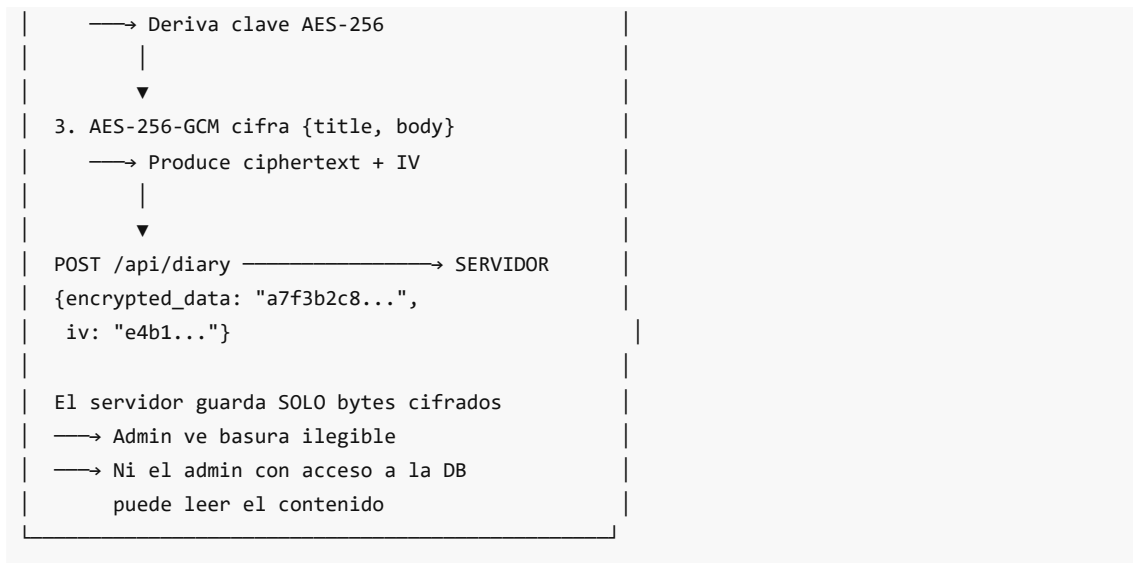
- **FCM Push** - Notificaciones push nativas (Firebase Cloud Messaging)
- **Email** - Notificaciones por correo electrónico vía Symfony Mailer
- **In-app** - Notificaciones dentro de la aplicación

3. Diario Personal Cifrado (NUEVO)

3.1 Arquitectura de cifrado

El Diario Personal es la funcionalidad más sensible de la plataforma, diseñada con **privacidad absoluta** como requisito principal.





3.2 Componentes del Sistema

Archivo	Rol	Líneas
src/Entity/DiaryEntry.php	Entidad Doctrine (user_id, encrypted_data, iv, timestamps)	57
src/Repository/DiaryEntryRepository.php	Repositorio con findByUser()	20
src/Controller/Api/DiaryController.php	5 endpoints REST + autenticación	139
migrations/Version20260627140251.php	Creación de tabla + columnas	63
assets/app.js (Diary module)	Módulo JS con cifrado/descifrado + UI	~200

3.3 Endpoints de la API

Método	Ruta	Función
GET	/api/diary	Listar entradas del usuario (cifradas)
POST	/api/diary	Crear nueva entrada cifrada
PUT	/api/diary/{id}	Actualizar entrada existente
DELETE	/api/diary/{id}	Eliminar entrada
GET	/api/diary/setup	Obtener token de verificación
POST	/api/diary/setup	Guardar token de verificación

Autenticación: Vía X-Game-Code header o sesión web. Solo el propietario puede leer/escribir sus entradas.

3.4 Detalle del cifrado (Web Crypto API)

```
// 1. Derivación de clave (PBKDF2)
const salt = 'TNSVT-DIARY-' + userCode;
```

```
const keyMaterial = await crypto.subtle.importKey('raw', password, 'PBKDF2', false,
['deriveKey']);
const key = await crypto.subtle.deriveKey(
  { name: 'PBKDF2', salt, iterations: 200000, hash: 'SHA-256' },
  keyMaterial,
  { name: 'AES-GCM', length: 256 },
  false,
  ['encrypt', 'decrypt']
);

// 2. Cifrado
const iv = crypto.getRandomValues(new Uint8Array(12));
const ciphertext = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, plaintext);

// 3. Almacenamiento (IV + ciphertext concatenados y base64)
const combined = new Uint8Array([...iv, ...ciphertext]);
const payload = btoa(String.fromCharCode(...combined));
```

Parámetros de seguridad:

- Algoritmo: AES-256-GCM (autenticado)
- Derivación: PBKDF2 con 200,000 iteraciones SHA-256
- Salt: prefijo fijo + código de usuario
- IV: 12 bytes aleatorios por cada cifrado
- La contraseña NUNCA se envía al servidor

3.5 Token de verificación

Para verificar que la contraseña ingresada es correcta SIN enviar la contraseña al servidor:

1. Al crear la contraseña: se cifra el texto conocido "TNSVT-DIARY-VERIFIED" y se almacena en el servidor
2. Al abrir el diario: se obtiene el token, se intenta descifrar con la contraseña ingresada
3. Si descifra correctamente → contraseña correcta
4. Si falla → contraseña incorrecta (el servidor nunca supo la contraseña real)

4. Autenticación Biométrica (Huella Digital)

4.1 Plugin Capacitor

Se utiliza el plugin `@aparajita/capacitor-biometric-auth` v10.0.0 que proporciona:

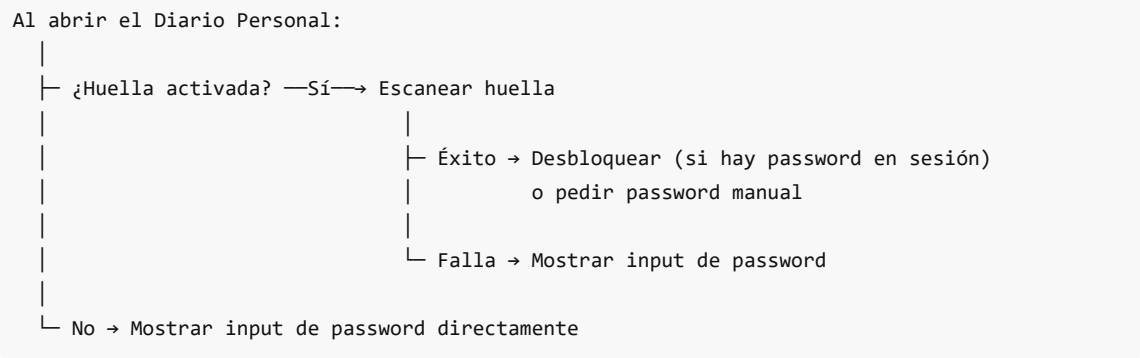
- Autenticación por huella digital (Android fingerprint)
- Autenticación facial (Face ID / Face Unlock)
- Verificación de disponibilidad del sensor
- Manejo de errores (cancelación, bloqueo, no disponible)

4.2 Integración

Componente	Archivo
Plugin npm	package.json → @aparajita/capacitor-biometric-auth@10.0.0
Permiso Android	AndroidManifest.xml → USE_BIOMETRIC

Módulo JS	assets/app.js → window.BiometricAuth (~40 líneas)
Botón UI	templates/base.html.twig → huella en estado bloqueado

4.3 Funcionamiento



Alcance actual: La huella desbloquea el diario solo si la contraseña ya fue ingresada en la sesión activa. En futuras versiones se puede extender para almacenar la contraseña cifrada con clave biométrica.

5. PWA (Progressive Web App)

5.1 Service Worker

Archivo: public/sw.js (v36)

Estrategias de caché:

Tipo de recurso	Estrategia
Páginas HTML	Cache-first con fallback a red
API calls (/api/...)	Network-first con fallback a caché
Assets con ?v=	Network-only (siempre fresco)
Assets estáticos	Cache-first

5.2 Manifest

Archivo: public/manifest.json

Propiedad	Valor
Nombre	T.N.S.V.T - Reino del Cristo Íntegro
Display	standalone
Orientación	portrait
Tema	#d4af37 (dorado)
Fondo	#0d0818 (violeta oscuro)

Iconos	192x192, 512x512, maskable-512x512
--------	------------------------------------

5.3 Instalación PWA (NUEVO)

Se agregó listener para el evento `beforeinstallprompt` que permite:

- Detectar cuando Chrome/Edge considera la app como instalable
- Mostrar un botón "📱 Instalar App" controlado por la aplicación
- Manejar el flujo de instalación programáticamente

6. Configuración de Red y Acceso

6.1 Tailscale

La aplicación se sirve a través de Tailscale, una VPN mesh zero-config:

Modo	URL	Acceso
Serve (actual)	<code>https://laptop-ebgqig6j.tailf43f87.ts.net</code>	Solo tailnet (dispositivos autorizados)
Funnel (opcional)	Misma URL	Público (cualquier persona en internet)

6.2 Servidor de desarrollo

```
php -S 0.0.0.0:8000 -t public
```

6.3 Construcción del APK

```
$env:JAVA_HOME="C:\dev\jdk\jdk-21\jdk-21.0.7+6"
cd android
.\gradlew.bat assembleDebug
```

7. Plugins Capacitor Instalados

Plugin	Versión	Propósito
@capacitor/app	8.1.0	Ciclo de vida de la app
@capacitor/network	8.0.1	Detección de conectividad
@capacitor/push-notifications	8.1.1	Notificaciones push nativas
@capacitor/splash-screen	8.0.1	Pantalla de carga
@capacitor/status-bar	8.0.2	Personalización de barra de estado
@aparajita/capacitor-biometric-auth	10.0.0	Autenticación biométrica (huella) ✓ NUEVO

8. Base de datos

8.1 Esquema

Tablas principales: `users`, `wallet_transactions`, `tournaments`, `tournament_entries`, `duels`, `duel_rounds`, `trades`, `feed_posts`, `liked_posts`, `notifications`, `devices`, `conversations`, `conversation_participants`, `messages`, `tasks`, `game_scores`, `academia_contents`, `module_progresses`, `trader_profiles`, `market_candles`, `monitor_events`, `diary_entries` (nueva)

8.2 Tabla `diary_entries`

```
CREATE TABLE diary_entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  encrypted_data CLOB NOT NULL,          -- Texto cifrado en base64
  iv VARCHAR(48) NOT NULL,              -- Vector de inicialización
  created_at DATETIME NOT NULL,
  updated_at DATETIME DEFAULT NULL,
  user_id INTEGER NOT NULL,              -- FK a users(id)
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Columnas agregadas a `users` :

- `diary_setup_token` CLOB DEFAULT NULL — Token de verificación cifrado
- `diary_setup_iv` VARCHAR(48) DEFAULT NULL — IV del token

9. Historial de Versiones

Versión	Código	Fecha	Cambios
1.5.0	10	27/06/2026	Diario Personal cifrado + Autenticación biométrica
1.4.0	9	27/06/2026	Tailscale HTTPS, calendar fallback, hub fix
1.3.1	8	25/06/2026	Mystic glow-up v3.6, calendar timeline
1.3.0	7	24/06/2026	1v1 Duel backend, game app fixes
1.2.0	6	21/06/2026	Mercure tiempo real, chart en vivo
1.1.0	5	17/06/2026	Wallet, torneos, Stage 1
1.0.0	4	10/06/2026	Versión inicial con chat, academia, feed

10. Repositorios

Proyecto	URL	Privado
Symfony (Backend + Web App)	https://github.com/federicocasal73-gif/tnsvt-symfony	Sí
Game App	https://github.com/federicocasal73-gif/tnsvt-market-instinct	Sí